

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA

PEDRO LOPES CAMPOS FERREIRA
EMANUELLE CAMILE DE PAULO LIMA
Repositório:
<https://github.com/pedrolopesengcomp/PT1_SO/tree/master>

Trabalho prático 1

SO

26/10/2026

1 Questão 1

Link para acesso ao arquivo fonte .c <https://github.com/pedrolopesengcomp/PT1_SO/blob/master/questao1/threads.c>.

As threads foram implementadas utilizando a biblioteca *pthread* do C. Todas as variáveis são do tipo *ponto flutuante* com uma **long** flag antes, para delimitar o valor do somatório parcial.

O somatório parcial foi implementado considerando que cada thread pegará um pedaço específico da soma e, assim, acelerando o processo de resultado. Foram definidas constantes do total de cálculos para 1 bilhão, de acordo com a determinação do problema. No final, o valor de cada soma parcial é somada para um total e multiplicada por 4. Os resultados do tempo de execução foram salvos em um arquivo CSV, que será lido por um script em python posteriormente e todos os gráficos serão gerados.

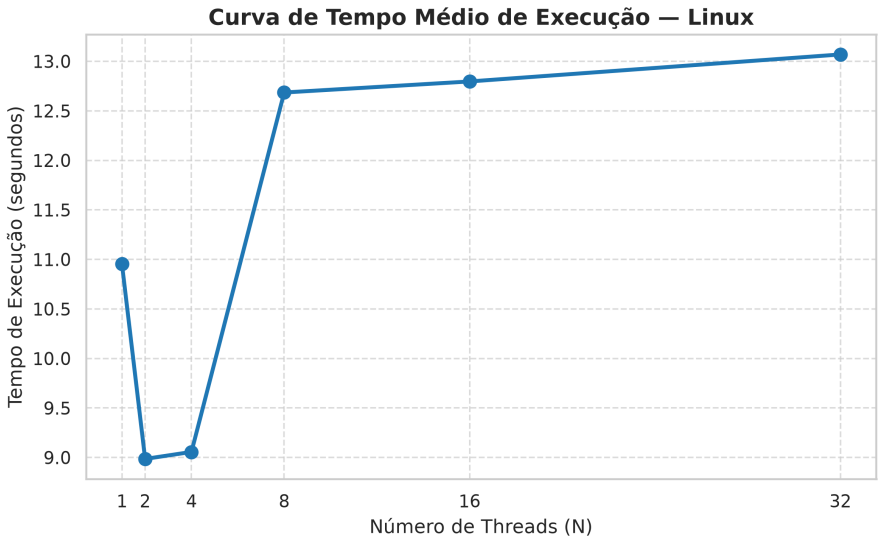


Figura 1 – Gráfico de execução do numero de threads X tempo

Ambos os computadores em que testamos foram em Linux com threads preempitvas e, como ambos foram executados em sistemas linux, o modelo de funcionamento é 1:1, como o funcionamento do Linux.

2 Questão 2

Link para acesso aos arquivos fonte .c da questão 2: <https://github.com/pedrolopesengcomp/PT1_SO/tree/master/questao2>.

Foram implementados dois processos independentes, A e B, que se comunicam por filas de mensagens POSIX. O processo A sorteia dois inteiros entre 0 e 99 e os envia a B; o processo B devolve a soma após aguardar 1 segundo. Os testes foram executados em Ubuntu sobre WSL 2 (Windows), com o código compilado pelo *gcc* e ligado à biblioteca *librt* (flag `-lrt`).

A solução foi feita em duas versões. A versão 1 segue o enunciado com o mínimo de filas possível, e a versão 2 corrige o problema observado quando há mais de um processo de cada tipo.

2.1 Decisões de projeto

Número de filas. Para um processo A e um processo B são necessárias duas filas: `/fila-tp1so-pedidos`, no sentido A → B, e `/fila-tp1so-respostas`, no sentido B → A. Uma única fila não serve, pois o próprio A poderia ler de volta o pedido que acabou de enviar. A versão 2 usa uma fila de pedidos compartilhada mais uma fila de resposta privada para cada processo A, ou seja, $1 + N$ filas para N processos A.

Formato das mensagens. Cada mensagem é uma *struct* de tamanho fixo. O pedido (`pedido_t`) contém `n1`, `n2` e o PID do A que fez o pedido; a resposta (`resposta_t`) contém `n1`, `n2`, `soma` e o PID do B que respondeu. A resposta repete `n1` e `n2` para que o A confira se ela corresponde ao seu pedido. Quando não corresponde, o A imprime a marcação `ERRADO!`, o que torna o problema mensurável.

Atributos das filas. As filas são criadas com `mq_maxmsg` = 10 e `mq_msgsize` igual ao tamanho exato da *struct*. Os dois processos abrem as filas com `O_CREAT`, de modo que a ordem de inicialização não importa. As chamadas `mq_send` e `mq_receive` são bloqueantes.

Encerramento por sinais. Os sinais `SIGINT` e `SIGTERM` são tratados com `sigaction`, sem a flag `SA_RESTART`. Assim, um `mq_receive` bloqueado retorna com `EINTR`, o laço principal termina, o A imprime o total de respostas recebidas e de respostas erradas, e as filas são fechadas. Isso permite encerrar os processos com Ctrl+C ou com o comando `kill`.

Limpeza. O programa `limpa_filas` remove as filas compartilhadas com `mq_unlink`, o que é útil quando algum processo é encerrado sem limpar. Na versão 2, cada A remove a própria fila de resposta ao terminar.

2.2 Metodologia dos testes

Os cenários com mais de um processo foram executados por 20 segundos cada, com o script `testar.sh`, que remove filas de execuções anteriores, inicia os processos B, inicia os processos A, aguarda 20 segundos e, por fim, envia `SIGTERM` aos processos A e depois aos B. O cenário com um A e um B foi executado manualmente em dois terminais, por cerca de 33 segundos. Em todos os casos, o número de respostas e de respostas erradas foi obtido do resumo que cada processo A imprime ao encerrar.

2.3 Resultados

A Tabela 1 resume os resultados. A vazão total é a soma das respostas de todos os processos A dividida pela duração do teste; os valores ficam um pouco abaixo de 1 e 2 por causa do tempo de início e de encerramento dos processos.

Tabela 1: Resultados dos cenários de teste

Versão	**Cenário**	**Duração**	**Respostas por A**	**Erradas por A**	**Vazão (resp./s)**
1	1 A, 1 B	$\approx$ 33 s	33	0	$\approx$ 1,0
1	2 A, 1 B	20 s	10 e 9	0 e 0	$\approx$ 0,95
1	1 A, 2 B	20 s	19	0	$\approx$ 0,95
1	2 A, 2 B	20 s	19 e 19	1 e 1	$\approx$ 1,9
2	2 A, 2 B	20 s	19 e 19	0 e 0	$\approx$ 1,9

2.4 Análise dos cenários

Um A e um B. O sistema produz uma resposta por segundo, limite imposto pela espera de 1 segundo do B. Não há como uma resposta ir para o processo errado, pois cada fila tem um único leitor.

Dois A e um B. O único B é o gargalo: ele atende os pedidos em ordem FIFO, alternando entre os dois A, e cada A recebe cerca de uma resposta a cada 2 segundos. Não houve erros nesta execução, mas isso é coincidência de escalonamento, e não uma garantia do programa. O A que espera há mais tempo no `mq_receive` costuma ser acordado primeiro e, como o B responde na mesma ordem dos pedidos, a resposta tende a cair no A certo. Se a ordem de chegada ao `mq_receive` se inverter, as respostas trocam de dono.

Um A e dois B. Os dois B se alternam no atendimento, mas a vazão continua em cerca de uma resposta por segundo. Como o A só tem um pedido pendente por vez, sempre há um B ocioso: acrescentar servidores não ajuda quando o cliente é síncrono.

Dois A e dois B. A vazão praticamente dobra, porque os dois B aguardam em paralelo, mas surgem respostas trocadas. No sétimo ciclo, o B de PID 2669 recebeu 92 e 16 do A de PID 2672, e o B de PID 2668 recebeu 89 e 49 do A de PID 2673. O B 2669 terminou primeiro e sua resposta foi lida pelo A 2673, que imprimiu $89 + 49 = 108$ (sendo que $108 = 92 + 16$); o A 2672 recebeu a resposta do outro pedido. A quantidade de trocas varia a cada execução, o que caracteriza uma condição de corrida.

2.5 Problema observado e solução

A causa das trocas é a fila de respostas compartilhada. Uma fila POSIX entrega a mensagem mais antiga de maior prioridade a quem chamar `mq_receive`, sem permitir que o leitor escolha mensagens por destinatário. Com dois A bloqueados na mesma fila, qualquer um deles pode retirar a resposta destinada ao outro.

A solução adotada na versão 2 dá a cada A uma fila de resposta privada, cujo nome inclui o PID do processo (`/fila-tp1so-resp-<PID>`). O A envia o próprio PID no pedido; o B abre a fila correspondente com `mq_open` sem `O_CREAT`, envia a soma e fecha a fila. Se o A já tiver terminado, o `mq_open` falha e o B descarta o pedido. A Figura 2 compara o fluxo das mensagens nas duas versões.

Versão 1: uma fila de respostas para todos os A



Qualquer A bloqueado em `mq_receive` pode retirar a resposta destinada ao outro A.

Versão 2: uma fila de resposta privada por A



O B escolhe a fila de resposta pelo PID que veio no pedido; cada A só lê a sua.

Figura 2 – Fluxo das mensagens nas versões 1 e 2

No cenário com dois A e dois B, a versão 2 teve 0 respostas erradas em 38 e manteve a vazão de cerca de 1,9 respostas por segundo.

Duas alternativas foram descartadas. Um semáforo que serializasse pedido e resposta também corrigiria o erro, mas eliminaria o paralelismo entre os processos B. Filas de mensagens System V resolveriam o problema com o campo `mtype` igual ao PID do destinatário, mas fogem do escopo das filas POSIX pedidas no trabalho.

2.6 Arquivos do código-fonte

- `comum.h`: nomes das filas, *structs* das mensagens, tratamento de sinais e abertura das filas;
- `processo_a.c` e `processo_b.c`: versão 1, com fila de respostas compartilhada;
- `processo_a2.c` e `processo_b2.c`: versão 2, com fila de resposta privada por A;
- `limpa_filas.c`: remove as filas compartilhadas;
- `Makefile`: compila tudo com `make`; `make limpar` remove as filas;
- `testar.sh`: executa um cenário, no formato `./testar.sh <versão> <nº de A> <nº de B> [segundos]`.